React Training — Day 2

Project Structure + JSX + JSX Rules

Goal of Today

- **React project structure:** Learn how Vite scaffolds and organizes your web application.
- **Role of main.jsx:** Understand the true entry point of a modern React app.
- **Role of App.jsx:** See how the main root UI component is rendered.
- **JSX concept:** Learn the declarative syntax that powers elegant React UIs.
- **JSX rules:** Master the required rules to write clean, error-free JSX.

React Project Structure

Vite Project Tree

A typical React project initialized with Vite:

```
react-learning
├── node_modules
├── public
├── src
│   ├── assets
│   ├── App.jsx
│   ├── main.jsx
│   └── index.css
├── index.html
└── package.json
```

The src Folder

This is the most important folder in your project directory.

All React code lives here. Later in the course, we will create reusable UI components inside this directory.

```
src
├── components
├── pages
├── assets
└── App.jsx
```

The Kitchen Analogy

Think of your React project like a busy restaurant kitchen 🍽️

- **node_modules:** Kitchen equipment & appliances.
- **public:** Ready-to-serve items.
- **src:** The main cooking area.
- **App.jsx:** The main signature dish.
- **main.jsx:** The restaurant entrance.
- **package.json:** The master recipe book.

Understanding main.jsx

The Entry Point

```
import React from "react"
import ReactDOM from "react-dom/client"
import App from "./App.jsx"

ReactDOM.createRoot(document.getElementById("root")).render(
  <App />
)
```

What it Does

`main.jsx` is the **entry point** of your entire React application.

It is responsible for two major tasks:

1. Loads the App component into memory.
2. Renders it directly inside the `index.html` DOM tree.

The Cinema Analogy

How pieces fit together

Think of the React rendering cycle like a cinema 🎬

- **App.jsx:** The Movie
- **main.jsx:** The Projector
- **index.html:** The Cinema Screen

The projector (main.jsx) streams the movie (App.jsx) directly onto the screen (index.html).

Understanding App.jsx

App.jsx controls exactly what appears on the screen. It is the primary, root component where your entire UI tree begins.

What is JSX?

JavaScript XML

JSX allows us to write HTML-like syntax inside JavaScript. It looks like HTML, but is compiled to JavaScript.

```
function App() {
  return (
    <h1>Hello Sauravh1>
  )
}
```

Why JSX Exists?

Without JSX, writing UI components is verbose and hard to read.

Without JSX:

```
React.createElement("h1", null, "Hello")
```

With JSX (Clean & Readable):

```
<h1>Helloh1>
```

JSX Rules (1-3)

Rule	❌ Incorrect	✅ Correct

1. One Parent Element
Must return a single root element
	`<h1>Hih1>`
	`<p>Welcomep>`
		`<div>`
		`  <h1>Hih1>`
		`  <p>Welcomep>`
		`div>`

2. Use className Instead of standard HTML 'class'	`<h1 class="title">Hih1>`	`<h1 className="title">Hih1>`
3. Close All Tags Even elements without children	`<img src="a.png">`	`<img src="a.png" />`

JSX Rules (4-5)

4. JavaScript Inside { }

You can dynamically inject any JavaScript variable or expression directly using curly braces.

```
function App() {
  const name = "Saurav";

  return <h1>Hello {name}h1>;
}
```

> Output: Hello Saurav

5. Expressions Allowed

Mathematical expressions, variables, and standard JS logic work seamlessly inside the brackets.

```
const a = 10;
const b = 5;

return <h1>{a + b}h1>;
```

> Output: 15

JSX Example Project

Let's put it together. Notice how we use variables and inline styling in JSX.

Note: Inline styling requires double curly braces {{ }} because a style is passed as a JavaScript object.

```
function App() {
  const name = "Saurav";
  const role = "Full Stack Developer";

  return (
    <div>
      <h1 style={{color: "cyan"}}>Portfolioh1>
      <h2>Name: {name}h2>
      <p>Role: {role}p>
      <button>Contact Mebutton>
    div>
  )
}
```

Practice Tasks

Developer Card

Create a UI with your Name, Skill, and Goal. Include a functional "Hire Me" button below.

Math in JSX

Display the result of a math operation: `10 + 5 = 15` strictly using JSX expressions.

Image Display

Render an image using a self-closing tag: without wrapper errors.

What's Next? (Day 3)

React Components

Learn to build and structure functional components.

Reusable UI

Stop repeating code. Build modular, reusable elements.

Imports & Exports

Manage your component structure across multiple files.

Profile Card

Build a dynamic, modular Profile Card Component System!

Questions?

You have successfully completed Day 2.

Get ready for ⚛️ React Components tomorrow!

Image Sources

https://i.etsystatic.com/36556407/r/il/7c9209/7173732405/il_570xN.7173732405_hgh1.jpg

Source: www.etsy.com

---

https://images.stockcake.com/public/9/e/4/9e4ecb47-b2c5-469f-a607-1636f7bb5b43_large/neon-cinema-projector-stockcake.jpg

Source: stockcake.com

---

https://static.vecteezy.com/system/resources/thumbnails/066/326/117/small/futuristic-circuit-board-with-glowing-blue-accents-and-central-copy-space-for-technology-designs-free-photo.jpg

Source: www.vecteezy.com